

14.1 Listas

Una **lista** es una estructura de datos que permite almacenar **una secuencia ordenada de elementos**, los cuales pueden ser de cualquier tipo: números, cadenas, booleanos, incluso otras listas. A diferencia de otros lenguajes que requieren declarar el tipo de dato, en Python una lista puede contener elementos **de distintos tipos**.

Las listas son **mutables**, lo que significa que pueden modificarse después de ser creadas: se pueden **agregar, eliminar o cambiar elementos**.

En Python, una lista se **declara utilizando corchetes []** y separando los elementos con comas. No es necesario especificar el tamaño ni el tipo de datos que va a contener.

```
lista_videojuegos = []  
lista_notas = []  
lista_estudiante = []
```

Esto crea una lista sin elementos, a la que luego se le pueden agregar valores dinámicamente con métodos como **append()**.

Podríamos también declarar listas con valores iniciales:

```
lista_videojuegos = ["Fortnite", "Apex", "AOE II", "Valorant"]  
lista_notas = [8.50, 6.10, 10, 4.80]  
lista_estudiante = ["Joaquín", "Perez", 27268, True]
```

Las listas pueden contener elementos:

- del **mismo tipo** (como en `lista_videojuegos` o `lista_notas`),
- o de **tipos distintos** (como en `lista_estudiante`), gracias a que Python es un lenguaje dinámico.

14.1.1.Métodos de Listas

Un **método** en Python es una **acción que se puede realizar sobre un objeto**, en este caso, una lista. Se escribe como el **nombre del objeto seguido de un punto y luego el nombre del método**.

Podés pensar en un método como una **función especial asociada a una lista**, que te permite **modificarla, consultarla o transformarla** de alguna manera.

Cuando usás un método, en realidad estás **llamando a una función que la lista invoca para manipular su contenido de alguna forma**. Estas funciones ya vienen **definidas en**

la **biblioteca estándar de Python**, por lo que **no necesitás escribirlas desde cero**: simplemente las llamás cuando las necesitás o cuando el programa lo requiere.

Método `append()`

El método `append()` se utiliza para **agregar un elemento al final de una lista**.

Cuando llamás al método `append()` sobre una lista, le pasás como argumento **un solo elemento**, y ese elemento se agrega como el **último** en la lista.

```
lista_videojuegos = ["Fortnite", "Apex", "AOE II", "Valorant"]
lista_notas = [8.50, 6.10, 10, 4.2]
lista_estudiante = ["Joaquín", "Perez", 27268, True]

lista_videojuegos.append("Warzone")
lista_estudiante.append(7.66)
```

En este ejemplo `lista_videojuegos` tenía 4 elementos y con `append()` agregamos otro más, lo mismo en `lista_estudiante`. Si quisiéramos imprimir por pantalla ambas listas:

```
['Fortnite', 'Apex', 'AOE II', 'Valorant', 'Warzone']
['Joaquin', 'Perez', 27268, True, 7.66]
```

Importante:

- `append()` **no devuelve una nueva lista**, sino que **modifica la lista original**.
- `append()` solo agrega **un elemento a la vez**. Si se pasa una **lista como argumento**, esa lista se agrega **como un único elemento**, es decir, **como una sublista dentro de la lista original**. Este comportamiento es similar al de una **matriz**, donde cada elemento representa una **fila** que, a su vez, es una lista con varias **columnas** (otros elementos). Así, se forman listas anidadas, es decir, listas dentro de listas.

```
lista_estudiante.append(lista_notas)
```

Cuando agregamos una **sublista** a una lista usando el método `append()`, esa sublista se agrega como **un único elemento dentro de la lista principal**, lo que quiere decir que **los corchetes de la sublista también se incluyen como parte de la lista principal**.

```
['Joaquin', 'Perez', 27268, True, [8.5, 6.1, 10, 4.2]]
```

En este caso, `lista_estudiante` contiene:

- **Elementos simples** como nombre, apellido, ID y un valor booleano,

- **Una sublista** con las notas del estudiante, que está dentro de sus propios corchetes.

Así, la sublista **[8.5, 6.1, 10, 4.2]** se agrega tal cual a `lista_estudiante`, y no se descompone ni separa en sus elementos individuales, sino que permanece como un único elemento dentro de la lista principal.

Cuando tenemos una lista anidada, podemos acceder a la sublista y usar **append()** para agregarle nuevos elementos. Para ello debemos primero ingresar en el índice de la sublista y luego utilizar el método:

```
lista_estudiante[4].append(9.5)
```

De esta forma, accedemos al índice 4 que es el quinto elemento de nuestra lista y recién en ese momento llamamos al método **append()**, enviando por parámetro la nueva nota.

```
['Joaquin', 'Perez', 27268, True, [8.5, 6.1, 10, 4.2, 9.5]]
```

Método insert()

El método **insert()** se utiliza para **insertar un elemento en una posición específica** dentro de una lista, **desplazando los elementos posteriores** a la derecha.

Supongamos que a `lista_estudiante` le queremos agregar la carrera que estudia antes del nombre:

```
lista_estudiante.insert(0, "TUP")
```

```
['TUP', 'Joaquin', 'Perez', 27268, True, [8.5, 6.1, 10, 4.2, 9.5]]
```

insert() recibe dos parámetros, el primero es el índice con el que quiero interactuar y el segundo es el elemento que quiero ingresar. De ésta forma nos queda como primero elemento TUP que es la carrera que estudia y el resto de los elementos los corre una posición hacia la derecha.

En caso de que quisiéramos ingresar un elemento con **insert()** a una sublista, al igual que con el método **append()** debemos ingresar en el índice donde se encuentra la sublista y recién ahí llamar al método **insert()**

```
lista_estudiante[5].insert(2, 8.0)
```

```
['TUP', 'Joaquin', 'Perez', 27268, True, [8.5, 6.1, 8.0, 10, 4.2, 9.5]]
```

Si el índice que se pasa es mayor que el número de elementos de la lista, el elemento se agregará al final de la lista.

Método extend()

El método **extend()** se usa para agregar múltiples elementos a una lista, uniendo otra secuencia (otra lista, tupla, etc.) al final de la lista original. A diferencia de **append()**, que agrega un único elemento (y si ese elemento es una lista, la agrega como sublista), **extend()** agrega los elementos uno por uno.

```
lista_videojuegos_2 = ["Resident Evil 3", "Mario Kart 8"]  
lista_videojuegos.extend(lista_videojuegos_2)
```

```
['Fortnite', 'Apex', 'AOE II', 'Valorant', 'Warzone', 'Resident Evil 3', 'Mario Kart 8']
```

Los elementos de **lista_videojuegos_2** se agregan por separado en **lista_videojuegos** y no como un bloque armando otra sublista.

Al igual que los métodos **append()** e **insert()** para acceder a sublistas debemos especificar el índice en el que se encuentran.

Método remove()

El método **remove()** se utiliza para **eliminar el primer elemento que coincida con el valor proporcionado** en una lista. Es importante aclarar que no se elimina por posición (como **pop()**), sino por **valor**.

```
lista_videojuegos.remove("AOE II")
```

El método **remove()** recibe como parámetro el valor del elemento que quiero eliminar de la lista, cuando encuentra la primera coincidencia lo elimina y los que seguían a él los mueve hacia la izquierda.

```
['Fortnite', 'Apex', 'Valorant', 'Warzone', 'Resident Evil 3', 'Mario Kart 8']
```

- Solo se elimina **la primera coincidencia** con el valor **'AOE II'**.
- Si hubiera más elementos iguales y quisiéramos eliminarlos también, hay que aplicar **remove()** nuevamente.

Si se intenta eliminar un valor que **no está presente**, se produce un **error**. Para que no falle el programa se puede validar de varias formas, un ejemplo es con un condicional:

```
elemento = "AOE II"

if elemento in lista_videojuegos:
    lista_videojuegos.remove(elemento)
else:
    print(f"{elemento} no es parte de la lista")
```

El operador **in** se utiliza para **verificar si un valor está contenido dentro de una secuencia**, como una lista, una cadena de texto (string), una tupla, un diccionario o un conjunto (set).

- Retorna **True** si el valor se encuentra dentro de la secuencia.
- Retorna **False** si el valor no está presente.

Método pop()

El método **pop()** elimina un elemento de la lista y devuelve el valor eliminado. Por defecto, elimina el **último elemento**, pero también se le puede indicar la **posición específica** que queremos eliminar.

- Si se usa **pop()** con un índice fuera del rango de la lista, se produce un error (**IndexError**).
- Si se usa sin argumentos, elimina el **último elemento**.

```
lista_videojuegos.pop()
```

- Es útil cuando se necesita **guardar el elemento eliminado**, ya que lo retorna.

```
elemento_eliminado = lista_videojuegos.pop()

print(f"Se elimino de la lista: {elemento_eliminado}")
```

```
Se elimino de la lista: Mario Kart 8
```

Método index()

El método **index()** busca un valor dentro de una lista y devuelve la **posición (índice)** de su **primera aparición**.

- Si el valor está presente, retorna el índice donde aparece por primera vez.
- Si el valor **no está en la lista**, lanza un error (**ValueError**).
- Si hay duplicados, no muestra las otras posiciones.

```
if "Valorant" in lista_videojuegos:  
    indice = lista_videojuegos.index("Valorant")  
    print(f"{lista_videojuegos[indice]} está en la posición {indice + 1}")
```

Con un condicional nos aseguramos de que si el elemento no está en la lista el programa no falle. Y, le sumamos una unidad a la variable indice para que cuente al elemento del índice 0 como 1.

```
Valorant está en la posición 4
```

El método **index()** tiene una forma extendida que permite indicar **dónde empezar** y **dónde terminar** la búsqueda dentro de la lista.

```
inicio = 1  
fin = 4  
indice = lista_videojuegos.index("Apex", inicio, fin)
```

Por ejemplo, podemos tener dos variables para indicar principio y fin y se las mandamos por parámetro al método **index()** para que use ese rango de búsqueda. Podemos enviar como parámetro solo el valor que deseamos buscar, el valor y el inicio o el valor, el inicio y el fin.

Si el valor **no aparece** dentro del rango dado, se lanza una **excepción ValueError**.

Método count()

El método **count()** **devuelve la cantidad de veces** que un valor aparece dentro de una lista. Es útil para saber la frecuencia de un elemento.

```
lista_videojuegos = ["Fortnite", "Apex", "AOE II", "Valorant", "Fortnite"]  
print(f"Fortnite aparece {lista_videojuegos.count('Fortnite')} veces en la lista")
```

```
Fortnite aparece 2 veces en la lista
```

Si el elemento no existe en la lista, el método **count()** devuelve cero.

Método clear()

El método **clear()** **elimina todos los elementos** de una lista, dejándola vacía. La lista sigue existiendo, pero queda sin contenido.

- No borra la lista como variable, solo **su contenido**.
- Después de usar **clear()**, el largo de la lista es 0.

```
lista_videojuegos.clear()  
print(lista_videojuegos)
```

```
[]
```

La lista queda vacía pero sigue existiendo la variable, por esto nos muestra el par de corchetes vacío.

Método reverse()

El método **reverse()** **invierte el orden** de los elementos de una lista **modificando la lista original**. No devuelve una nueva lista, sino que altera directamente la que se aplica.

La lista original **se modifica en el momento**.

```
lista_videojuegos = ["Fortnite", "Apex", "AOE II", "Valorant", "Fortnite"]  
lista_videojuegos.reverse()  
print(lista_videojuegos)
```

```
['Fortnite', 'Valorant', 'AOE II', 'Apex', 'Fortnite']
```

Método sort()

El método **sort()** se utiliza para **ordenar los elementos de manera ascendente**, es decir, **modifica la lista original y no devuelve nada**.

```
lista_videojuegos = ["Fortnite", "Apex", "AOE II", "Valorant", "Fortnite"]  
lista_videojuegos.sort()  
print(lista_videojuegos)
```

```
['AOE II', 'Apex', 'Fortnite', 'Fortnite', 'Valorant']
```

¿Qué tipos de datos se pueden ordenar?

- Números: de menor a mayor.
- Cadenas: orden alfabético (según Unicode).
- No se puede ordenar una lista que mezcle tipos incompatibles (como strings y números).

En caso de querer ordenar la lista de manera descendente se le debe enviar por parámetro lo siguiente: `reverse = True`


```
lista_videojuegos = ["Fortnite", "Apex", "AOE II", "Valorant", "Fortnite"]  
lista_videojuegos.sort(reverse = True)  
print(lista_videojuegos)
```

```
['Valorant', 'Fortnite', 'Fortnite', 'Apex', 'AOE II']
```

14.1.2. Instrucción del

del no es un método, sino una **instrucción de Python** que se usa para **eliminar elementos de una lista** (o eliminar la lista entera si se desea). A diferencia de métodos como **pop()** o **remove()**, **del** permite borrar elementos **por índice**, **por rango**, o incluso **toda la lista completa**.

Eliminar un solo elemento por índice

```
lista_videojuegos = ["Fortnite", "Apex", "AOE II", "Valorant", "Fortnite"]  
del lista_videojuegos[1]  
print(lista_videojuegos)
```

```
['Fortnite', 'AOE II', 'Valorant', 'Fortnite']
```

Eliminar un rango de elementos (slicing)

```
lista_videojuegos = ["Fortnite", "Apex", "AOE II", "Valorant", "Fortnite"]  
del lista_videojuegos[1:3]  
print(lista_videojuegos)
```

```
['Fortnite', 'Valorant', 'Fortnite']
```

Eliminar toda la lista

```
lista_videojuegos = ["Fortnite", "Apex", "AOE II", "Valorant", "Fortnite"]  
del lista_videojuegos
```

Si usás **del** sobre la lista completa, ya no va a existir en memoria. Es decir que la se elimina la variable.

11. Arreglos, Arrays bidimensionales

En la programación, un **array bidimensional** (también conocido como **matriz**) es una estructura de datos que nos permite almacenar información en forma de tabla. Es decir, se trata de una colección de **filas y columnas**, donde cada elemento puede ser accedido utilizando **dos índices**: uno para la fila y otro para la columna.

11.1. Matrices

Podemos pensar en una hoja de cálculo o en una tabla de Excel, donde cada celda contiene un dato y tiene una posición definida por una fila y una columna. Por ejemplo, si tuviéramos una tabla de personas donde cada fila representa una persona y cada columna un dato (nombre, edad, DNI, etc.), estaríamos usando una matriz.

Características principales de una matriz

- Todos los elementos deben ser del mismo tipo (enteros, strings, etc.).
- El tamaño (cantidad de filas y columnas) **se define al inicio** y no cambia durante el programa (para simular un array estático).
- El primer índice representa la **fila**, y el segundo índice representa la **columna**.
- Al igual que en los arrays unidimensionales, **los índices comienzan en 0**.

En Python, podemos pensar los **arrays bidimensionales** como **listas anidadas**, es decir, cada **fila** de la matriz es una **lista individual**. Siguiendo con el ejemplo de una tabla de personas, cada fila representaría a una persona, y cada columna contendría un dato específico como el **nombre**, la **edad**, el **DNI**, etc.

Esto es posible porque en Python **no existen matrices estáticas como en otros lenguajes**, sino que son **estructuras dinámicas**, lo cual brinda mayor flexibilidad. Además, Python permite que dentro de una misma matriz se puedan utilizar distintos tipos de datos (por ejemplo, strings y enteros combinados), a diferencia de otros lenguajes en los que se debe respetar un único tipo de dato para todos los elementos del array.

Declaración

Aunque Python no tiene arrays estáticos como en otros lenguajes, podemos **simular una matriz estática** utilizando listas anidadas, es decir, una lista de listas.

Por ejemplo, si queremos declarar una matriz de 3 filas y 4 columnas, inicializada con ceros, podemos hacerlo así:

```
matriz_personas = [[0] * 4 for persona in range(3)]
```

- **[[0] * 4]** crea una **fila** con 4 elementos, todos inicializados en 0.
- **for _ in range(3)** repite esa fila **3 veces**, una por cada fila de la matriz.

De esta forma estamos creando una **matriz de 3x4**:

3 filas → 3 listas internas

4 columnas → cada lista interna tiene 4 elementos

Entonces, si lo pensamos como tabla tendríamos una estructura así:

	[0]	[1]	[2]	[3]
[0]				
[1]				
[2]				

Acceso a los índices para carga de datos

Una vez que tenemos nuestra matriz creada (por ejemplo, de 3 filas y 4 columnas), podemos **recorrerla con un doble ciclo for** para permitir que el usuario cargue cada posición.

```
filas = 3
columnas = 4
matriz_personas = [[0] * columnas for persona in range(filas)]

print("Ingrese los datos para la matriz:")
for i in range(filas):
    for j in range(columnas):
        matriz_personas[i][j] = input(f"Ingrese el valor para la posición [{i}][{j}]: ")
```

- Recorremos la matriz con dos **for**:
El externo para las filas (**i**), el interno para las columnas (**j**).
- En cada posición **matriz[i][j]**, se solicita al usuario un número.
- Usamos **input(...)** para asegurarnos que se guarde el dato ingresado

Si quisiéramos **asignar un valor** a una posición en particular de una matriz, podemos hacerlo **accediendo directamente al índice de la fila y de la columna** mediante sus números literales, es decir, escribiéndolos explícitamente entre corchetes.

```
matriz_personas[1][0] = "Edgardo"
```

En este caso, estamos accediendo a la **segunda fila ([1])** y a la **primera columna ([0])**, y asignando el valor **"Edgardo"** a esa posición.

Recordemos que:

- Los índices en Python comienzan desde 0.
- Por lo tanto, `matriz_personas[1][0]` representa el **primer dato de la segunda persona** en nuestra matriz simulada.

Este tipo de asignación es muy útil cuando queremos **modificar datos ya cargados** o **cargar información puntual en una posición específica** de la matriz.

Muestra de datos

```
print("\nDatos ingresados:")
for i in range(filas):
    for j in range(columnas):
        print(f"Índice [{i}][{j}]: {matriz_personas[i][j]}")
    print() # Salto de línea al terminar cada fila
```

- `print(f"Índice [{i}][{j}]: {matriz_personas[i][j]}")`: muestra cada dato mostrando cada índice
- `print()` sin argumentos al final de cada fila hace el salto de línea para que los datos se vean en forma de tabla.

Si quisiéramos acceder a un único elemento de la matriz, podemos hacerlo **especificando directamente el índice de la fila y el índice de la columna** entre corchetes, utilizando sus valores literales (es decir, los números directamente).

```
matriz_personas[1][0]
```

Estaríamos accediendo al **primer elemento de la segunda fila**. Recordemos que en Python los índices comienzan en 0, por lo tanto:

- `matriz_personas[0]` es la **primera fila**
- `matriz_personas[1]` es la **segunda fila**
- Dentro de esa fila, `[0]` indica el **primer elemento** (la primera columna)

Entonces `matriz_personas[1][0]` se interpreta como: *"elemento de la segunda fila y primera columna"*.

Este tipo de acceso es útil cuando sabemos exactamente **en qué posición** se encuentra el dato que queremos consultar o modificar.